

Great question — this is exactly the right moment to **slow down, read deliberately, and restructure with intent** rather than “tweak until it works”.

Below is a **learning-driven, implementation-oriented plan** tailored to **your actual template structure** and your goal: *learn CSS, Sass, HTML, Jekyll, Ruby, and Gems by revising and combining files consciously*.

I’ll assume the **three files** you’re referring to are:

1. `template_structure-v1.txt` (full project tree)
2. `sass-learning-map-and-design-review.md`
3. `sass-learning-map-and-review-plan.md`

I’ll explicitly anchor steps to your real folders and files.

I’ll also frame each step as **what to read, what to revise, and what to combine**.

0. Mental Model (Read This First)

Before touching files, lock this model in your head:

```
Ruby + Gems
  ↓
Jekyll (build system)
  ↓
Liquid templates + Markdown
  ↓
HTML structure
  ↓
Sass (SCSS preprocessing)
  ↓
CSS (final output)
```

Your goal is **not** to master everything at once, but to understand **how data and decisions flow downward**.

Phase 1 — Read the Project as a System (No Editing Yet)

1. Read the template structure *top-down*

Start with the tree in `template_structure-v1.txt` and annotate it mentally like this:

Layer	Folder	Purpose
Build config	<code>_config.yml</code> , <code>Gemfile</code>	Ruby + Jekyll behavior
Page structure	<code>_layouts</code> , <code>_includes</code>	HTML + Liquid
Content	<code>_posts</code> , <code>.md</code> , <code>.html</code>	Markdown → HTML
Styling source	<code>_sass</code> , <code>assets/css/main.scss</code>	Sass architecture
Output	<code>_site/</code>	Generated (DO NOT EDIT)

📌 **Rule:** `_site/` is *read-only knowledge*, never a place to learn by editing.

This phase is purely about orientation.

👉 Source: project structure

Phase 2 — Sass: Learn by Reading in the Correct Order

Your Sass structure already follows a **professional ITCSS-like hierarchy**. That's a gift — don't fight it.

Read order (VERY important)

Read files in **this exact order**, even if it feels boring:

1 `_sass/0-settings/`

- `_colors.scss`
- `_global.scss`

What you learn here

- Design tokens
- Variables as *decisions*, not styles
- Why nothing here should output CSS directly

✍️ Revision task:

- Rewrite comments in your own words.
- Add comments explaining *why* a variable exists, not what it is.

2 **_sass/1-tools/**

- `_mixins.scss`
- `_grid.scss`
- `_normalize.scss`
- `_reset.scss`

What you learn

- Sass as a *language* (functions, mixins)
- CSS normalization vs reset
- DRY principles (like Python helpers)

✍️ Revision task:

- For each mixin, write:

```
// Usage:  
// @include ...  
// Produces: ...
```

3 **_sass/2-base/_base.scss**

What you learn

- Element selectors
- Typography defaults
- The difference between *structure* and *components*

✍️ Revision task:

- Identify:
 - HTML element selectors (`body` , `p` , `a`)
 - Utility decisions vs visual ones

4 **_sass/3-modules/**

- `_buttons.scss`
- `_footer.scss`
- `_pagination.scss`

What you learn

- Component-level thinking
- Naming conventions
- Reusability

✍️ Revision task:

- Rename comments to:

```
// Component: Button
// Variants: primary, secondary
```

5 **_sass/4-layouts/**

- `_home.scss`
- `_page.scss`
- `_post.scss`
- `_tags.scss`

What you learn

- Page-specific overrides
- Why layouts come *after* modules

✍️ Revision task:

- Add a comment at top:

```
// Layout scope: applies only to X layout
```

6 `_sass/5-trumps/_helpers.scss`

What you learn

- Specificity control
- Utility classes
- When breaking purity is acceptable

👉 Revision task:

- Flag any class you think is dangerous with a comment.

Phase 3 — Understand How Sass Is Combined

`assets/css/main.scss` is the *entry point*

This file answers one key question:

“In what order does CSS get built?”

👉 Exercise:

- Rewrite the import list into commented sections:

```
// 0. Settings
// 1. Tools
// 2. Base
...
```

This is where **you eventually plug in your Persian palette system** cleanly.

📌 Do NOT add logic here yet — only structure.

Phase 4 — HTML & Jekyll: Map CSS to Markup

Now that you understand styles, you connect them to **where they apply**.

Read order (HTML/Liquid)

1. `_layouts/default.html`
2. `_includes/head.html`
3. `_includes/header.html`
4. `_layouts/page.html`
5. `_layouts/post.html`

What you're learning

- Layout inheritance
- How CSS classes enter the DOM
- How Jekyll composes pages

✍️ Exercise:

- For every major CSS class you saw in Sass, find:
 - Which layout/include introduces it
 - Which page uses it

This is where Sass suddenly makes sense.

Phase 5 — Ruby & Gems (Minimal but Essential)

You don't need Ruby mastery — you need **literacy**.

Files to read:

- `Gemfile`
- `Gemfile.lock`

What to understand

- Gems = dependencies (like `requirements.txt`)
- Why Jekyll version is pinned

- Why GitHub Pages is special

✍️ Exercise:

- Add comments explaining:
 - Why `jeekyll` is pinned
 - What happens if versions drift

Phase 6 — Combine & Refactor (Your First Real Change)

Only **after** all reading and commenting:

Safe first refactor

1. Create a new folder:

`_sass/0-settings/palette/`

2. Move:

- Persian palette
- Light/Dark semantic mappings (as you planned)

3. Update `main.scss` imports **only**

🎯 Goal:

- No visual change
- Cleaner mental model
- Zero regressions

Phase 7 — Feedback Loop (How You Learn Fast)

For each change:

1. Compile Sass **alone** (you already discovered `sass CLI`)
2. Commit small changes

3. Push → let GitHub Pages validate Jekyll

Why This Plan Works for You

- Matches **your learning-map docs**
- Treats Sass like Python modules, not magic
- Separates **reading**, **rewriting**, and **design decisions**
- Prevents premature refactors

Next Step (Concrete)

If you want, next we can:

- Walk **one file together** (e.g. `_colors.scss`)
- Rewrite its comments *line by line*
- Decide exactly where Persian palette + dark mode logic lives

Just tell me which file you want to start with.